Ders 7 Genişletilmiş Notu: Veri Sıkıştırma, İletişim Hataları (Parity, Hamming) ve Algoritmaya Giriş

Ders 7 Genişletilmiş Notu: Veri Sıkıştırma, İletişim Hataları (Parity, Hamming) ve Algoritmaya Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Veri Sıkıştırmaya Dönüş
- 2. Run-Length Encoding (RLE)
- 3. Frekansa Bağlı Kodlama
- 4. Göreli Kodlama
- 5. Sıkıştırmanın Kazançları ve Ağ Üzerinde Şifreleme
- 6. Ham Veri
- 7. Sıkıştırma Kullanan Formatlar
- 8. OCR: Resimden Metin Elde Etmek
- 9. Sözlük Kodlaması
- 10. İletişim Hataları
- 11. Eşlik Biti (Parity Bit)
- 12. Hamming Uzaklığı
- 13. Hamming Uzaklığıyla Hata Düzeltme
- 14. Algoritma Nedir?
- 15. Algoritmanın Tanımı ve Kritik Noktaları
- 16. Algoritmayı İfade Etme Yolları
- 17. Akış Diyagramı Sembolleri
- 18. Akış Diyagramında Kullanılan İşlemler
- 19. Örnek 1: İki Sayıyı Toplayan Algoritma
- 20. Örnek 2: Dört İşlem Yapan Algoritma
- 21. Sonraki Derse Bırakılan Problem
- 22. Ders Sonunda Gelen Soru: Sıkıştırma ve Şifreleme

1. Veri Sıkıştırmaya Dönüş

Hoca, bugün ilk kez algoritma konusuna gireceğimizi, ama önce önceki derste yarım kalan veri sıkıştırmayı tamamlayacağımızı söyledi. Önceki ders kayıplı ve kayıpsız sıkıştırma kavramlarında bitmişti. Bu yüzden konunun üzerinden bir kez daha geçildi.

1.1 Sıkıştırmanın Amacı

Veri sıkıştırma (data compression), herhangi bir verinin daha az yer kaplayarak saklanmasını sağlayan bir çözümdür. Sıkıştırılan şeyin dosya olması gerekmez. Hocanın bu noktadaki düzeltmesi de buydu: söz konusu olan “herhangi bir veri”dir.

Hocanın sayısal örneği şöyleydi. 100 KB’lık bir veri sıkıştırıldıktan sonra bellekte 80 KB yer kaplıyor olsun. Tasarruf şöyle hesaplanır:

$$\frac{100-80}{100} = \frac{20}{100} = \%20$$

Bu oran sıkıştırılan veriye göre değişir. Her verinin içindeki örüntü farklı olduğu için tek bir yöntem yoktur. Önemli olan, verinin örüntüsüne uygun bir sıkıştırma algoritması kullanmaktır. Hocaya göre yöntemlerin çoğu uzun yıllardır kullanılıyor. Yeni yöntemler ortaya atılsa da alan çok hızlı değişmiyor, çünkü yaklaşımlar genel olarak oturmuş durumda.

1.2 Kodlama, Kod Çözme, Kayıpsız ve Kayıplı Sıkıştırma

Sıkıştırmada iki işlem uygulanır:

- **Kodlama (encoding):** Veri bir sıkıştırma algoritmasından geçirilir.
- **Kod çözme (decoding):** Sıkıştırılmış veri çözülerek önceki veri elde edilmeye çalışılır.

İki tür arasındaki fark kod çözmenin sonucuna bakılarak anlaşılır. Kod çözme sonunda verinin aynısı elde ediliyorsa **kayıpsız sıkıştırma (lossless compression)** söz konusudur. Aynı veri elde edilemiyorsa, yani bazı bilgiler eksik kalıyorsa **kayıplı sıkıştırma (lossy compression)** söz konusudur.

1.3 Neden Her Dosya Baştan Sıkıştırılmış Tasarlanmıyor?

Soru: Bir programın kullandığı dosya formatı sıkıştırılabiliyorsa, program neden o dosyayı baştan sıkıştırılmış biçimde kullanacak şekilde tasarlanmıyor? **Cevap:** Hoca soruyu sınıfa yöneltti ve bir öğrencinin ek işlem yükü cevabını onayladı. Her sıkıştırma ve her açma işleminde işlemci gücü harcanır. Veri bir algoritmadan geçirilerek sıkıştırılır, ama sıkıştırılmış hâli okunduğunda bir şey ifade etmez. Veriye tekrar ulaşmak için önce kod çözme yapmak gerekir, bunun da bir maliyeti vardır. Bu süre algoritmaya göre değişir, kimi algoritmada daha kısa, kimisinde daha uzundur.

1.4 Aynı Kavram Başka Yerde: Veri Güvenliği

Hoca, kodlama ve kod çözmenin bilgisayar mühendisliğinde başka nerede kullanıldığını sordu. Cevap şifrelemeydi. Veri güvenliğinde (data security) kodlama ve kod çözme çok sık kullanılır ve şifreleme ile şifre çözme adımları olarak geçer.

Genel tanım şöyledir: bir veriyi alıp başka bir formata, farklı bir saklanma biçimine dönüştürmeye kodlama, tekrar eski hâline getirmeye kod çözme denir. Hoca, şifrelemeyle ilgili birkaç algoritmik örneğin algoritma konusu ilerledikçe görüleceğini de söyledi.

2. Run-Length Encoding (RLE)

İlk yöntem Run-Length Encoding'dir. Bir öğrenci fikri şöyle özetledi: üç tane A varken bir A yazıp yanına tekrar sayısını koyabiliriz. Hoca bunu onayladı. Saklanan bilgi tekrar ediyorsa değer bir kez, tekrar sayısıyla birlikte yazılır. Bunu "3A" ya da "A3" diye yazmak fark etmez, çünkü yöntemin çeşitli uygulamaları vardır.

Hoca iki örnek üzerinden ilerledi: bir metin bilgisi ve bir resmin piksel değerleri.

2.1 Piksel Örneği

Verilen: Resmin renk değerleri `10 10 10 10 255 255 255 0 0 0 0 5`.

Hoca bunun kaç byte yer kapladığını sordu. Sınıf önce değerleri bit dizisi sandı ve 22 ya da 24 gibi cevaplar verdi. Hoca kendi yazımının da bu karışıklığa yol açtığını söyledi. Karışıklığı çözen ayrım şudur: dizide yalnız 0 ve 1 olsaydı ikilik düzende bitler olarak okunabilirdi, ama 5 gibi başka değerler varsa böyle okunamaz.

Adım 1, orijinal boyut: Her değer 1 byte içinde saklanabilir. 10, 255, 0 ve 5'in hepsi 1 byte ile ifade edilir. 12 değer olduğu için boyut 12 byte'tır.

Adım 2, tekrarları sayma: 4 tane 10, 3 tane 255, 4 tane 0, 1 tane 5 vardır.

Adım 3, sıkıştırılmış hâl: `4 10 3 255 4 0 1 5`. Burada 8 değer olduğu için boyut 8 byte'tır.

Adım 4, kazanç: 4 byte kazanılmıştır.

$$\frac{12-8}{12} = \frac{4}{12} \approx \%33$$

2.2 Metin Örneği

Verilen: `AAABBCCCDDEEF`, her karakter 1 byte olduğu için 13 byte.

Sıkıştırılmış hâl `3A2B3C2D2E1F`, yani 12 byte'tır. Yalnızca 1 byte kazanılmıştır:

$$\frac{13-12}{13} = \frac{1}{13} \approx \%7{,}7$$

Sınıf ve hoca sonucu $1/13$ olarak bıraktı, yüzde değeri sonradan yazıldı (ara adım eklendi).

Hoca bu verimliliğin oldukça düşük olduğunu söyledi.

2.3 RLE Ne Zaman İşe Yarar, Ne Zaman Zarar Verir?

Tekrar fazlaysa yöntem iyi sonuç üretir. Özellikle resimlerde gökyüzü, çimenlik alan veya deniz gibi büyük bir alanı kaplayan, renkleri birbirine yakın bölgelerde bu yaklaşımı kullanmak akıllıca görünür.

Tekrar sık değilse yöntem ek maliyet getirir. Hocanın örneği `ABCDEF` idi. Her harf tek geçtiği için her biri "1 harf" olarak yazılır. Böylece 6 byte'lık veri 12 byte olur ve bilgi gereksiz yere iki kat yer kaplar.

> **Hocanın vurgusu:** Bu önemli. Sıkıştırma algoritması seçerken hangi veriyi sıkıştırdığınıza mutlaka bakmak gerekir.

2.4 Soru-Cevap

Soru: Sıkıştırılmış hâlde 8 byte'ı nasıl saydık? **Cevap:** 4, 10, 3, 255, 4, 0, 1, 5. Yani 4 tane 10, 3 tane 255, 4 tane 0 ve 1 tane 5.

Soru: Bilgisayar `4103255...` gibi yan yana yazılmış değerleri okurken "3 255" mi "32 55" mi olduğunu nasıl anlıyor? **Cevap:** Böyle bir ayrıma ihtiyaç yoktur. Değerler yalnızca kâğıtta yan yana görünür. Bellekte her biri kendi byte'lık hücresinde durur: 4 belleğin bir gözünde, 10 bir sonraki gözünde, 3 ondan sonrakinde, 255 de ondan sonrakindedir. 3 bir yerde, 255 başka bir yerde saklandığı için karışıklık oluşmaz.

```
Bellek gözleri:  | 4 | 10 | 3 | 255 | 4 | 0 | 1 | 5 |
```

3. Frekansa Bağlı Kodlama

İkinci yöntem frekansa bağlı kodlamadır (frequency dependent encoding). Değişken uzunluklu kodlama (variable-length encoding) da denir.

Hoca alfabeden yola çıktı: Türkçede en sık hangi harfler kullanılır? Sınıf a ve e gibi sesli harfleri söyledi. Yumuşak g, ü ve ş gibi harfler ise diğerlerine göre daha az kullanılır.

Bir öğrenci, tamamlanabilecek sesli harflerin atılabileceğini önerdi. Hoca bunun doğru olmadığını, ama iyi bir deneme olduğunu söyledi. Başka bir öğrencinin önerisi doğru yöndeydi: her harf için bir byte kullanmak yerine a için daha az bit kullanılabilir.

Yöntemin mantığı şudur. Sık kullanılan harfler daha az bitle, seyrek kullanılan harfler daha çok bitle kodlanır. Hocanın örneğinde a ve e için 3 bit kullanılabilir, az kullanılan harfler için daha fazla bit kullanılır. Metinde sık geçen harfler az bitle ifade edildiği için toplamda yerden tasarruf edilir. Hoca bunu bir dengeleme olarak anlattı.

Ders sonrası not: Hoca burada kullanım sıklığının “şifrelemeyi” yapmamızı sağladığını söyledi. Bağlam sıkıştırma olduğu için doğru sözcük “kodlama”dır.

Bu yaklaşımı benimseyen algoritmalardan biri Huffman kodlamasıdır. Hoca, Huffman kodlamasının bu dersin konusu olmadığını, sonraki dönemlerde görüleceğini söyledi.

4. Göreli Kodlama

Üçüncü yöntem göreli kodlamadır (relative encoding). Fark kodlaması (differential encoding) olarak da geçer ve daha çok videolarda kullanılır.

Hoca nasıl çalıştığını sordu. Bir öğrenci, az hareketli saniyelerde kare sayısının düşürülebileceğini söyledi. Hoca böyle yaklaşımlar olduğunu, ama bu yöntemin bu olmadığını belirtti. Başka bir öğrenci, iki kare arasının tahmin edildiğini önerdi. Hoca iki karenin kullanıldığını kabul etti, ama “tahmin” demeyi doğru bulmadı. Doğru cevap, benzer parçaların önceki kareden, yeni eklenen parçaların yeni kareden alınmasıydı.

Hocanın anlatımıyla iki sahne düşünelim. Film çok hareketli değilse, örneğin ağır işleyen bir dramsa, art arda gelen karelerin çoğunda aynı sahneyi görürsünüz. İki kare arasındaki farka bakılır. Karşı tarafa yalnızca değişen kısımlar gönderilir. Sabit kalan pikseller önceki kareden korunur. Böylece çok daha az veri saklanır ve bilgi ağ üzerinden daha az veriyle gönderilir.

5. Sıkıştırmanın Kazançları ve Ağ Üzerinde Şifreleme

Hoca sıkıştırmanın sağladığı kazançları şöyle sıraladı:

- **Daha az yer:** Sakladığımız bilgi daha az yer kaplar.
- **Bant genişliği (bandwidth):** Veri aktarımında kullanılabilen yolun genişliği sınırlı olabilir. Bant genişliği düşükken gönderilen veri büyükse trafikte sıkışma ve gecikme yaşanabilir. 100 KB yerine 80 KB göndermek hız açısından da avantaj sağlar.
- **Maliyet:** Ev ve cep telefonu kotaları artık çok yükselmiş olsa da, sık veri gönderen sistemlerde veriyi küçültmek kotayı daha verimli kullanmak ve maliyeti düşürmek demektir.

Bu nedenle sıkıştırma, ağ üzerinden gönderilen veride çok sık kullanılır.

Ağ üzerinde kodlama ve kod çözmenin önemli olmasının bir nedeni de güvenliktir. Bir öğrencinin dediği gibi, başkasının veriyi okuması böylece zorlaşır. Hoca, verilerin artık genelde düz metin (plain text) olarak, yani şifrelenmeden gönderilmediğini söyledi. Şifreleme çoğu uygulamada uygulamanın kendisi ya da işletim sistemi seviyesinde yapılır, karşı tarafta da çözülür.

6. Ham Veri

Hoca ardından ham verinin (raw data) ne olduğunu sordu. Bir öğrenci, verinin kullanıcıdan alınmış, işlenmemiş saf hâli dedi. Hoca “sadece kullanıcı demeyelim” diye düzeltti: veri nereden elde ediliyorsa, işlenmemiş hâline ham veri denir.

RLE örneklerindeki üç A ile başlayan ve dört tane 10 ile başlayan satırlar verinin ham hâliydi. Altındaki sıkıştırılmış satırlar ise onun işlenmesiyle elde edildi. İşler genellikle ham veri üzerinden yapılır. Ham veride hatalı bilgiler de olabilir ve bunları düzeltmek için çeşitli yöntemler uygulanır.

7. Sıkıştırma Kullanan Formatlar

Hoca, sıkıştırma uygulanmış hangi formatların bilindiğini sordu. Sınıf RAR, MP3, FLAC, ISO, JPEG, ZIP, GIF ve 7z’yi saydı. Hoca bunların en çok bilinen formatlar olduğunu söyleyerek ikiye ayırdı:

- **Genel sıkıştırma formatları:** RAR, ZIP ve 7z, farklı dosya tiplerini sıkıştırıp saklamak için kullanılır. İçlerinde daha iyi sıkıştırma ya da daha hızlı sıkıştırma gibi seçenekler sunarlar. Seçim, sıkıştırılacak verinin miktarına bağlı olarak yapılır.
- **Özelleşmiş formatlar:** MP3 sesle, JPEG ve GIF resimle ilgilidir. Hoca bunlara MPEG ve MKV’yi de ekledi.

Ders sonrası not: Sınıfın saydığı ISO, hoca tarafından düzeltilmedi. ISO büyük olasılıkla bir sıkıştırma formatından çok disk kalıbı biçimidir. Hocanın saydığı MKV de büyük olasılıkla sıkıştırma algoritmasının kendisi değil, sıkıştırılmış ses ve görüntüyü taşıyan bir kapsayıcı formattır.

Hoca, yeni formatların daha çok hangi veriler için üretildiğini de sordu. Cevap, boyutları ve kaliteleri sürekli artan video ve resim, yani multimedya dosyalarıydı. Metin için de zaman zaman yeni yaklaşımlar ortaya konur.

8. OCR: Resimden Metin Elde Etmek

Hoca konuyla bağlantılı bir genel kültür sorusu sordu: size taratılmış bir dosya verilmiş olsun, bunu metin bilgisine nasıl dönüştürürsünüz? Cevap, Optik Karakter Tanıma (Optical Character Recognition, OCR) yöntemidir. Yöntem şu adımlarla çalışır:

- Resimdeki harfler tek tek tespit edilir.
- Boşluklar kullanılarak art arda gelen harflerin bir kelime oluşturduğu anlaşılır.
- Satır boşluklarından satırlar anlaşılır.
- Böylece resimden doğrudan metin bilgisi elde edilir.

9. Sözlük Kodlaması

Hoca sıkıştırma konusunu son bir yöntemle kapattı: sözlük kodlaması (dictionary encoding). Genel prensip, saklama işlemini sözlükteki kelimeler üzerinden yapmaktır.

Hocanın örneği "balıkesir" idi. Bu kelimede "balık" kısmı vardır. "Balık" bilgisine zaten sahipseniz üzerine "esir" eklemek yeterlidir. Bir kelime başka bir kelimenin içinde geçiyorsa bu bilgi kullanılır. Yani kelimeler, içlerindeki daha küçük kelime blokları kullanılarak saklanır.

Hoca bu yöntemin ve diğer algoritmaların ayrıntısına girmedi. Bu aşamada hedef, sıkıştırmanın ne amaçla kullanıldığını ve hangi yaklaşımların bulunduğunu genel olarak öğrenmekti.

10. İletişim Hataları

Aradan sonra hoca kısa bir konuya geçti: iletişim hataları (communication errors). Bir öğrencinin tanımını hoca beğendi: bilgisayarın iki parçası ya da iki farklı sistem arasındaki iletişimde oluşan hatalar. Yani bilgisayarın parçaları veya bilgisayarlar arasında veri aktarılırken oluşan hatalardan söz ediyoruz.

Hoca hatanın kendisinin ne olduğunu aradı. Öğrenciler önce donanım uyumsuzluğu, işletim sistemi farkı, kısa devre ve elektrik kesintisi gibi nedenler önerdi. Hoca bunların kaynak olabileceğini, ama asıl sorunun verideki hatanın ne olduğu olduğunu söyledi. Doğru cevap şuydu: sıfırlar bir, birler sıfır olmuş olabilir.

En temel hâliyle hata, ikili düzendeki verinin bitlerinin değişmesidir. **Bir bit bile değişse veri tamamen başka bir anlama gelebilir** ve bu da bütün sistemin yanlış çalışmasına yol açar.

Hatanın kaynakları ve oluştuğu yerler için hocanın saydıkları şunlardır:

- Elektrikten ya da elektronik devredeki bir parçadan kaynaklanan hatalar.
- İki birim arasındaki aktarım: bellekten CPU'ya, sabit diskten belleğe.
- İki bilgisayar arasındaki aktarım. Özellikle kablosuz ortamda fiziksel bir engel ya da radyo frekansındaki bir sorun hataya yol açabilir.

Hoca sonra kapsamı biraz genişletti. Hata yalnız haberleşmede değil, saklama ve yazma sırasında da olur. Ama CPU'dan belleğe yazmak da aslında bir haberleşme sayılır.

İletişim hataları denince iki başlık öne çıkar:

- **Hata tespiti (error detection):** Hatanın olduğunu bulmak. Görece kolay ve az maliyetlidir.
- **Hata düzeltme (error correction):** Hatayı hem bulmak hem düzeltmek. Daha maliyetlidir.

11. Eşlik Biti (Parity Bit)

Hata tespitinin en kolay yolu eşlik bitidir. Eşlik biti veriye eklenen ek bir bittir (additional bit). Değeri, verideki 1'lerin sayısına göre seçilir.

11.1 Tek Eşlik Örnekleri

Hocanın örneklerinde istenen, eklenen bitle birlikte toplam 1 sayısının tek olmasıydı.

Birinci örnek. Veri `01000001` (8 bit).

- Verideki 1 sayısı: 2.
- Toplamın tek olması için eşlik biti 1 olmalıdır.
- Sonuç: `1 01000001`. Toplam 1 sayısı 3'tür, yani tektir.

İkinci örnek. Veri `01000110` (8 bit).

- Verideki 1 sayısı: 3.
- Toplam zaten tektir, bu yüzden eşlik biti 0 olur.
- Sonuç: `0 01000110`. Toplam 1 sayısı yine 3'tür.

Hoca iki durumda da tahtaya şu anlamda bir ifade yazdı: toplam desenin (total pattern) tek sayıda 1'i vardır (odd number of ones).

11.2 Tek Eşlik ve Çift Eşlik

Eşlik biti iki türlü kullanılır:

- **Tek eşlik (odd parity):** Eklenen bitle toplam 1 sayısı tek olur.
- **Çift eşlik (even parity):** Eklenen bitle toplam 1 sayısı çift olur.

Böylece 8 bitlik veri taşınırken bir bit daha taşınır. Hoca bu biti koruma biti, yani hatayı tespit etme biti olarak adlandırdı.

11.3 Hata Nasıl Anlaşılır, Hangi Hata Kaçar?

Sistem hatayı 1'lerin sayısından anlar. Tek eşlikte 1 sayısı tek olmalıyken bir bit dönüşürse sayı çifte döner ve hata ortaya çıkar.

Hoca hangi hataların yakalanıp hangilerinin yakalanamadığını sordu. Bir öğrenci, sayı değişirse hatanın anlaşılacağını, ama bitlerin dizilişindeki hatanın anlaşılamayabileceğini söyledi. Asıl sonuç şuydu: bir bit değişirse hata anlaşılır, iki bit değişirse anlaşılamaz.

Soru: "İki bit" derken birinin 1'den 0'a, diğerinin 0'dan 1'e dönmesi mi gerekir? Örneğin üç tane 1'i olan veride iki tane 0, 1 olursa sistem bunu anlayabilir mi? **Cevap:** Hoca soruyu ikinci örnek üzerinden sınıfa sordu. İki sıfır 1 olursa 1 sayısı $3 + 2 = 5$ olur (ara adım eklendi). Sayı yine tektir ve eşlik biti 0 hâlâ doğru görünür. Bu yüzden sistem hatayı anlayamaz. Bir bit ya da üç bit değişseydi anlardı.

Sonuç olarak **çift sayıda bit değişirse hata fark edilmez**. Hocanın ifadesiyle bu yöntem, genelde bir bitin zarar gördüğü düşünülerek ortaya atılmış basit bir hata tespit yöntemidir.

11.4 İlgili Kavramlar

Hoca, eşlik bitlerinin oluşturduğu check byte kavramının da bulunduğunu, bununla özellikle sabit diskte oluşabilecek hatalar için karşılaşılabileceğini söyledi. Ayrıca döngüsel artıklık denetimi (cyclic redundancy check, CRC) ve sağlama toplamı (checksum) kavramlarını andı. Bunlar dersin birebir konusu değildir. Hoca genel kültür için bakılmasını önerdi.

> **Ek bilgi (derste anlatılmadı):** CRC ve checksum da eşlik biti gibi veriye ek denetim bilgisi koyarak hatayı tespit eder. Checksum büyük olasılıkla veri parçalarından basit bir toplam üretir. CRC ise büyük olasılıkla veriyi ikili bir bölme işlemine sokup kalanı denetim bilgisi olarak kullanır. Her ikisi de büyük olasılıkla tek eşlik bitine göre çok daha fazla hata türünü yakalar.

12. Hamming Uzaklığı

Hoca, tespitin yeterli olmadığını, mümkünse hatayı bulduktan sonra düzeltmek istediğimizi söyledi. Bunun için önce Hamming uzaklığı (Hamming distance) kavramı gerekir.

Hoca tahtaya iki bit deseni yazıp aralarındaki uzaklığı tahmin etmelerini istedi. Sınıftan 4, 2 ve 1 tahminleri geldi. Doğru cevap 2'ydi. Desenlerin bir konumunda 0 ile 1, bir konumunda 1 ile 0 farklıydı, kalan konumlar aynıydı.

Tanım: İki bit deseni (bit pattern) arasındaki Hamming uzaklığı, birbirinden farklı olan bit sayısıdır.

Hoca, literatürde Hamming uzaklığının başka işlemler için de kullanıldığının görüleceğini ekledi.

13. Hamming Uzaklığıyla Hata Düzeltme

13.1 Kod Tablosu

Hoca 8 sembol için bir kodlama tablosu verdi:

Sembol	Kod
A	`000000`
B	`001111`
C	`010011`
D	`011100`
E	`100110`
F	`101001`
G	`110101`
H	`111010`

Hoca iki gözlem istedi. Birincisi, 8 sembolü ifade etmek için 3 bit yeterlidir. Burada ise 6 bit kullanılmıştır, yani 3 bit fazladan harcanmıştır.

İkinci gözlem, ekstra bitlerin amacıdır. Bir öğrenci önce her iki sembol arasındaki farkın 4 olduğunu, sonra en az 3 olduğunu söyledi. Sınıf B ile C arasında ve A ile C arasında uzaklığın 3 olduğunu buldu. Sonuç şudur: **her iki sembol arasındaki Hamming uzaklığı en az 3'tür.** Kodlama bu özellik garanti edilerek yapılmıştır.

13.2 Hatalı Desenin Düzeltilmesi

Hoca bu kez karakter, kod, alınan desen ve uzaklık sütunlarından oluşan bir tablo çizdi.

Verilen: Alıcıya `010100` deseni gelsin.

Bu desen tablodaki hiçbir sembole denk gelmez. Yani bir hata olduğu anlaşılır. Ama hangi sembole en yakın olduğu bulunabilir.

Adım 1: Desen her kodla bit bit karşılaştırılır. Hoca A, B, C ve D'yi sınıfla birlikte yaptı, E, F, G ve H için yalnızca sonuçları söyledi. Farklı konumların dökümü (soldan sayılarak) ara adım olarak eklendi.

Sembol	Kod	Alınan desen	Farklı konumlar	Uzaklık
A	`000000`	`010100`	2, 4	2
B	`001111`	`010100`	2, 3, 5, 6	4
C	`010011`	`010100`	4, 5, 6	3
D	`011100`	`010100`	3	1
E	`100110`	`010100`	1, 2, 5	3
F	`101001`	`010100`	1, 2, 3, 4, 6	5
G	`110101`	`010100`	1, 6	2
H	`111010`	`010100`	1, 3, 4, 5	4

Derste bir öğrenci C satırı için sondaki `011`in `100`e dönüştüğünü, başka bir öğrenci de D satırı için soldan üçüncü bitin 1'den 0'a döndüğünü gösterdi.

Adım 2: En küçük uzaklık seçilir. En küçük değer 1'dir ve D sembolüne aittir.

Sonuç: Gönderilen bit grubu aslında D olacaktı, ama hatadan dolayı hiçbir sembole denk gelmedi. Desen D olarak düzeltilir ve yola devam edilir.

Hamming uzaklığı üzerinden hata düzeltme kabaca böyle yapılır. Gelen kod alınır, en yakın olduğu sembol bulunur ve o sembol olarak işaretlenir.

13.3 Bedeli

Düzeltmenin bir maliyeti vardır, çünkü 3 bit yerine 6 bitlik alan kullanılmıştır. Hoca şunu da belirtti: 6 bitlik kodun bellekteki karşılığını, 8'in katlarıyla nasıl çalışıldığını bu aşamada düşünmeyin. Önemli olan yaklaşımın felsefesi ve nasıl uygulandığıdır.

13.4 Soru: Aynı Kodu İki Kez Yazmak

Soru: Üç bitlik bir kod yerine aynı kodu iki kez yazsak, örneğin `001` yerine `001 001`, `111` yerine `111 111` kullansak, bir kopya bozulunca diğeri tutar diye hata düzeltme yapabilir miyiz? **Cevap:** Hoca böyle bir yöntem bilmediğini söyledi. Ama benzer bir fikrin disklerde bulunduğunu anlattı. İkincil belleklerde RAID düzenleri kullanılır, bunlar RAID 0 ile başlayıp 1, 2, 3 diye devam eder. Bu düzenlerde alanın bir kısmı, veri bozulma ihtimaline karşı tekrar eden (redundant) bilgi saklamaya ayrılır. Örneğin 100 GB'lık bir diskin 40 ya da 50 GB'ı efektif olarak kullanılır, kalanı bilginin yeniden kurtarılması için ayrılır. Disk bozulduğunda bilgi ayrılan alandan geri kazanılır. Soruda önerilen yapı için hoca olumsuz bir cevap da vermedi, sadece biraz maliyetli göründüğünü söyledi.

> **Ek bilgi (derste anlatılmadı):** Sorudaki yöntem büyük olasılıkla tekrar kodu (repetition code) olarak bilinir. İki kopya tek bitlik hatayı tespit eder, ama düzeltemez. Çünkü sorudaki `001 001` deseninde tek bit bozulursa iki kopya farklı çıkar, hangisinin bozuk olduğu ise bu fark ile anlaşılamaz. Üstelik 3 bitlik kodlardan yalnız bir biti farklı olan ikisi iki kez yazılırsa aralarındaki uzaklık 2 olur, yani ders tablosundaki en az 3 şartı sağlanmaz.

13.5 Tespit mi, Düzeltme mi?

Tespit önemlidir, çünkü hata bulununca uyarı verilebilir. Örneğin karşı bilgisayardan gelen bilgi hatalıysa "bir daha gönder" denebilir. Bu birimler düzeyinde de yapılabilir. Bilgiyi yeniden almak daha maliyetliyse bilgiyi biraz daha büyük taşıyıp hatayı olduğu yerde düzeltmek de bir çözümdür. Hoca ikisinin ihtiyaca göre tercih edildiğini söyledi.

13.6 Soru-Cevap

Soru: İki sembolün de uzaklığı 1 çıksaydı ne yapardık? Böyle bir ihtimal var mı? **Cevap:** Hoca bunun mümkün olmaması gerektiğini, tasarımın buna göre yapıldığını söyledi. Bir öğrenci gerekçeyi şöyle tamamladı: kodlar arası uzaklık en az 3 ise, desen bir koda 1 uzaklıktaysa diğerlerine en az 2 uzaklıktadır. Hoca bunu onayladı.

> **Ders sonrası not:** Hoca ardından "tek sayı olmasının arkasında yatan sebeplerden biri de bu" dedi. Asıl belirleyici olan uzaklığın tek olması değil, büyük olasılıkla en az 3 olmasıdır. Bu tabloda kod çiftlerinin uzaklıkları 3 ve 4'tür (Python ile denetlendi) ve her iki durumda da tek bitlik hata doğru düzeltilir.
>
> **Ek bilgi (derste anlatılmadı):** Bu kural literatürde büyük olasılıkla kodun en küçük uzaklığı (minimum distance) üzerinden ifade edilir. En küçük uzaklık 3 ise bozulan desen doğru koda 1, öbür kodlara en az 2 uzaklıkta kalır. Ders tablosundaki bütün kodlarda her tek bit bozulması Python ile denendi ve her seferinde doğru sembol bulundu. İki bitlik hatada ise bu garanti kalkar, çünkü desen yanlış bir koda daha yakın düşebilir ya da birkaç koda eşit uzaklıkta kalabilir.

Soru: A'nın kodunun ikinci bitini 1 yapsak, o da alınan desene 1 uzaklığa düşmez mi? **Cevap:** Düşmez, çünkü A'nın kodunu değiştirmiyoruz. A'nın kodu `000000` olarak bellidir ve kodlar birbirinden en az 3 bit farklı olacak şekilde seçilmiştir. Öğrenci bunun bir standart olduğunu sandı. Hoca bunun standart olmadığını, bu tablonun yalnızca bu özelliği sağlayan bir kodlama olduğunu, başka kombinasyonlarla da yapılabileceğini söyledi.

14. Algoritma Nedir?

Hoca bu konuyu "bizim için çok çok önemli" diye tanıttı. Şimdiye kadar bilgisayarın genel parçaları, donanım ve yazılım terimleri, bilgilerin nasıl saklandığı, dosya türleri, sayı düzenleri, düzenler arası geçiş ve lojik kapılar görüldü. İşletim sistemleri ve bilgisayar ağları daha sonraki bir haftada konuşulacak. Bu noktadan sonra soru şudur: bir bilgisayara komutlar üzerinden nasıl iş yaptırırız?

Hoca algoritmanın tanımını sınıfa sordu. Cevaplar şunlardı: bir sorunu çözmek için ortaya atılan çözüm yolu, belirli bir amaca ulaşmak için tasarlanan yol, olabildiğince optimize edilmiş çözüm yöntemi, adımları belli bir sıraya dizip bilgisayara anlatmak. Hoca bir cevapta geçen "çok detaylı" ifadesini kritik bulmadı. Önemli olan, bunun bir tarif olmasıydı.

14.1 Algoritma Yalnız Bilgisayara Özgü Değildir

Algoritma bilgisayarla çok ilişkilendirilse de sadece bilgisayara özgü bir kavram değildir. Günlük hayatta bir problemi çözmek ya da bir işi yapmak için uyguladığımız adımlar vardır. Bu adımların hepsine birden algoritma deriz. Derste geçen örnekler şunlardır:

- **Zeka küpü:** Önce bir kısmı tek renk yapılır, sonra belli algoritmalar üzerinden devam edilir. Hedef, her yüzeyde aynı rengin olmasıdır.
- **Kek ya da yemek tarifi:** Tarif aslında bir algoritmadır.
- **Alışveriş:** Önce liste oluşturulur, sonra birlikte yaşanan kişilere ihtiyaç sorulur. Uygun alışveriş yapılmak isteniyorsa ürünlerin nerede daha uygun olduğu bulunur, liste buna göre gözden geçirilir ve hangi ürünün nereden alınacağına karar verilir. Sonra rota belirlenir, ürünler alınır ve eve dönülür. Adımlar hedefe göre değişebilir, algoritma basit ya da karmaşık olabilir.
- **Mobilya yapımı:** Ahşap bir üründe önce ağaç alınır, işlenir, şekil verilir, boya sürülür, vernik atılır.

14.2 Bir Problemin Tek Bir Algoritması Yoktur

Hoca, bir işi yapmanın tek bir yolu olup olmadığını sordu. Cevap hayırdı. Her mobilyacının farklı bir yöntemi vardır. Bazı adımlar genelde aynı olsa da, örneğin boyayı ya da boyanın kullanılma şeklini değiştirerek fark yaratılabilir. Bir öğrenci bunu "Her yiğidin bir yoğurt yiyişi vardır" diye özetledi. Hoca bunu esprili ama yanlış olmayan bir cevap olarak değerlendirdi.

Hoca sonra COVID-19 örneğini verdi. Sağlık Bakanlığı virüs ilk çıktığında bir protokol belirledi, ama zaman içinde bu protokol farklılaştı. İlaçların uygulanma zamanı, sırası ve hangi aşamada hangi ilacın verileceği sürekli güncellendi. Hedef aynıydı, yani kişiyi kurtarmak. Daha iyi bir yöntem bulununca algoritma güncellendi. Hoca sağlık alanında her vakanın ayrı bir durum olduğunu, aynı yöntemin her zaman aynı sonucu vermediğini, bu yüzden sağlıkla örnek vermenin biraz tehlikeli olduğunu da ekledi. Örneği yalnız günlük hayattan olduğu için verdiğini söyledi.

Bilgisayar dünyasında da durum aynıdır. Ortaya attığınız bir yöntemden daha iyisini bulabilirsiniz. Yeni yöntem daha hızlı çalışabilir veya daha az yer kaplayabilir. Bir soru verildiğinde sınıfta en az 5-10 farklı çözüm görülecektir ve bunda bir sorun yoktur.

> **Hocanın vurgusu:** İlk hedef bir problemi çözmektir, verimli çözmek ikinci hedeftir. Dönemin en azından yarısına, belki üçte ikisine kadar hedef, verilen girdi için istenen çıktıyı üreten, yani doğru çalışan bir yöntem ortaya koymaktır.

15. Algoritmanın Tanımı ve Kritik Noktaları

Hoca İngilizce tanımı tahtaya yazdı ve sınıftan yorumlamasını istedi. Tanım şöyledir:

> “An algorithm is an ordered set of unambiguous, executable steps that defines a terminating step.”
>
> **Ders sonrası not:** Bu tanımın kaynak kitaplardaki biçimi büyük olasılıkla “… that defines a terminating process” şeklindedir. Algoritma tek bir sonlanan adım değil, sonlanan bir süreç tanımlar. Anlatımın geri kalanı da bu anlamdadır.

Yani algoritma; sıralı, belirsizlik içermeyen, uygulanabilir ve sonlanan adımlar kümesidir. Hoca bu noktaları tek tek açtı.

15.1 Belirlilik (Unambiguous)

“Ambiguous” belirsiz, “unambiguous” belirli demektir. Doğal dillerde bir ifade birden fazla anlama gelebilir. Bir cümle kurarsınız, kimi öyle anlar, kimi böyle. Bilgisayar dünyasında böyle bir yaklaşım yoktur. Bütün programlama dillerinde ve algoritmayı ifade ederken kullanılan notasyonlarda bir ifadenin tek bir anlamı vardır.

Bu yüzden tarif edilen algoritmada belirsizliğe götürebilecek bir ifade olmamalıdır. Algoritmayı sözel olarak anlatırken de şüpheye yer bırakmayan ifadeler kullanılmalıdır.

Soru: “Suyu bardağa dök” dediğimizde taşırmadan mı dökecek, bu tür bir kesinlikten mi söz ediyoruz? **Cevap:** Suyun bir miktarı varsa bunu ifade etmelisiniz: “100 ml su dök”. Sadece “su dök” demek, işin ne doğrultuda yapılacağını söylemez. Kişi su dökecek, ama ne kadar dökeceğini bilmeyecektir. Aynı şekilde bir işlemde 5 eklenmesi gerekiyorsa bunu söylemezseniz hata oluşur.

15.2 Uygulanabilirlik ve Makul Sürede Bitme

Bir öğrenci algoritmanın uygulanabilir olması gerektiğini söyledi. Hoca bu noktada algoritmanın polinom zamanda sonlanmasının önemli olduğunu belirtti. Bir algoritma istenen zamanda, yani bir saatte, iki saatte ya da birkaç günde bitmiyorsa, seneler sürecekse ve sonu görülemiyorsa o algoritmada bir problem vardır. Böyle bir şeyi algoritma olarak birinin önüne süremezsiniz.

> **Ders sonrası not:** Hoca polinom zamanda sonlanmayı algoritma olmanın şartı gibi anlattı. Algoritmanın tanımı büyük olasılıkla yalnız sonlu sürede bitmeyi ister. Polinom zamandan daha uzun süren yöntemler de büyük olasılıkla algoritma sayılır, ama büyük girdilerde pratikte kullanışsız kalabilir. Hocanın asıl vurgusu, pratikte makul sürede bitme beklentisidir.

15.3 Sonlanma

Her algoritma mutlaka sonuçlanmalıdır. Bir tarifin sonunda bir yere varılamıyor, olay ortada kalıyorsa ya da farklı kollardan gidilip bir noktaya ulaşılamıyorsa bir hata vardır. Her algoritmanın bir başlangıcı ve bir bitişi olmak zorundadır.

15.4 Doğruluk

Tasarlanan algoritmanın doğru olduğu da kontrol edilmelidir. Bakılacak şey, verilen girdi için üretilmesi gereken çıktının üretilip üretilmediğidir. Bir algoritma için tek bir durum (case), başka bir algoritma için üç durum olabilir. Bunlar tek tek kontrol edilmelidir.

Hoca bunu kek tarifiyle bağladı. Herkes farklı bir tarif verebilir ve biri diğerinden bir tık önde olabilir. Ama günün sonunda fırından yenilebilir bir ürün çıkmalıdır.

Hoca ayrıca ekran görüntüsü almakta sakınca olmadığını, ama notların temize çekilerek düzgün bir defterde toplanmasını önerdi.

16. Algoritmayı İfade Etme Yolları

Hoca bilgisayar biliminde algoritmaların hangi yollarla ifade edilebileceğini sordu. Sınıfın cevaplarıyla şu liste oluştu:

- **Programlama dilleri (programming languages):** C, Python, C++, Java. Arduino önerildiğinde hoca bunun bir programlama dili değil, bir platform olduğunu belirtti.
- **Akış diyagramı (flowchart notation):** Karşılaştırma, girdi gibi işler için semboller kullanılarak görsel olarak ifade edilir.
- **Sözde kod (pseudocode):** Kod ile konuşma dili arasında bir notasyondur. Kodun sadeleştirilmiş hâli olarak da düşünülebilir. Derste ileride anlatılacak.
- **Sözel anlatım (oral presentation):** Bir kişiye algoritmayı sözlü olarak tarif etmek. Sınıf az önce bunu yaptı. Yazılı anlatım da sayılabilir, ama çok sık kullanılmaz.

Bir öğrenci ilk bilgisayarlardaki delikli kartları ve kablolu programlamayı da önerdi. Hoca bunların artık kullanılmayan, çok eskide kalan yöntemler olduğunu ve bir bakıma o dönemin programlama yaklaşımı sayılabileceğini söyledi, listeye almadı.

Bu derste öncelik sırası bellidir: önce akış diyagramı, sonra C. Algoritma programlama diliyle ifade edilebilse de tasarım işi daha çok akış diyagramı ve sözde kodla yapılır.

> **Hocanın vurgusu:** Bu ders boyunca önce akış diyagramı, sonra kod yazılacak. Aksi belirtilmedikçe yalnız kod yeterli sayılmaz, çünkü önemli olan önce algoritmayı tasarlamaktır.

17. Akış Diyagramı Sembolleri

Hoca sembolleri sınıfa sorarak sıraladı:

- **Başlangıç ve bitiş (START, STOP):** Bir algoritmanın olmazsa olmazı, başlangıç ve bitiş noktasıdır.
- **Girdi (input):** Hoca bu kutuyu kulağı kesilmiş bir dikdörtgen olarak tarif etti. İçine `A` yazmak, A değişkenine dışarıdan okuma yapılacağı anlamına gelir.
- **Çıktı (output):** Girdinin karşılığında üretilen sonuçtur. İçine `B` yazmak, B değişkeninin ekranda kullanıcıya gösterileceği anlamına gelir.
- **İşlem (process):** Toplama, çıkarma ve atama gibi işlemler burada yapılır. Bir kutuda birden fazla işlem yazılabilir.
- **Karar (if):** Bilgisayarın bir yol seçmesini sağlar. Bir **doğru (true)** kolu, bir de **yanlış (false)** kolu vardır.
- **Ok işaretleri:** Bütün kutular arasındaki bağlantıyı gösterir.
- **Bağlantı numaraları:** Sayfa ya da yer biterse bir yerde `1` yazılarak bitirilir, kaldığınız yerde yine `1` yazılarak devam edilir.

Hoca Türkçe karşılıkları da yazdı. İkisini birlikte kullanmamayı, sadece ne olduklarını bilmeyi istedi.

Soru: Mantıksal işlemler (Boolean operations) için ayrı bir kutu gerekir mi? **Cevap:** Gerekmez, bunlar işlem kutusunda yapılır. Asıl ihtiyaç, karar verme işlemi için bir kutudur.

17.1 If Kutusunun Kuralı

Hocanın örneği şuydu: A 0'dan büyükse yapılacak iş true kolunda yapılır, değilse false kolunda. Bir işi bir kez yapıyorsanız, tekrar eden bir iş değilse if kutusu kullanılır.

> **Ders sonrası not:** Altyazıda false kolu için "A 0'dan küçük değilse" geçiyor. Koşul $A > 0$ olduğuna göre doğrusu "A 0'dan büyük değilse"dir, yani $A \leq 0$ durumu false koluna gider.
>
> **Hocanın vurgusu:** Burası önemli. True kolu olmayan, yalnız false kolu olan bir if tasarlanmaz. Böyle bir if yanlış tasarlanmıştır ve koşulun ters yazılması gerekir.

Soru: True kolu olup false kolu olmayan bir tasarım yapılabilir mi? **Cevap:** Evet. İş true kolundadır, false kolu boş bırakılır. Bu çok sık yapılan bir şeydir. Örneğin A 0'dan büyükse bir şey yapılacak, değilse hiçbir şey yapılmayacaksa false kolu boş kalır.

18. Akış Diyagramında Kullanılan İşlemler

Hoca örneklerden önce kullanılacak işlemleri tahtanın altına not etti:

- **Aritmetik işlemler:** Toplama, çıkarma, çarpma, bölme, `mod` (kalan) ve tam sayı bölmesi için `div`.
- **Karşılaştırma işlemleri (relational operators):** Büyüktür, küçüktür, büyük eşittir, küçük eşittir, eşittir ve eşit değildir.
- **Atama işlemi (assignment):** Bir değeri bir değişkene atar.

Atama örnekleri şunlardır:

- `B = A`
- `C = 5`, yani C'ye 5 atanır.
- `A = A + B`, yani A'nın şu anki hâli B ile toplanıp yine A'nın üzerine yazılır.

  Ders sonrası not: Hoca ilk satırı anlatırken "B değişkeni değerine A'ya atıyorsunuz" dedi. `B = A` yazımında atanan değişken soldadır, yani A'nın değeri B'ye atanır.

18.1 Eşittir İşaretinin İki Anlamı

Soru: Burada eşittir kullanıyoruz. If'in içinde karşılaştırma, başka yerde atama olarak mı alınıyor? **Cevap:** Evet. İşlem kutusundaki eşittir atama içindir. Karar kutusundaki eşittir, iki ifadenin birbirine eşit olup olmadığını kontrol eder. Programlama dillerinde bu farklı ifade edilir. Örneğin C'de karşılaştırma için iki eşittir (==), atama için bir eşittir (=) kullanılır. Akış diyagramında kontrol için == yazmak da yanlış sayılmaz.

19. Örnek 1: İki Sayıyı Toplayan Algoritma

İlk örnek, iki sayıyı toplayan algoritmanın akış diyagramıdır. Hoca bunu sınıfla kurdu.

Bir öğrenci START'tan sonra sayıların okunmasını önerdi. Hoca okumadan önce yapılması gereken bir iş olduğunu söyledi. Başka cevaplardan sonra doğru cevap geldi: algoritmayı kullanacak kişiye ne yapacağı açıkça söylenmelidir. Bu yüzden önce çıktı kutusuna bir mesaj yazılır. Mesaj yazılmazsa kullanıcı ne girmesi gerektiğini bilemez. Hoca bunun program yazarken de mutlaka kullanılacağını söyledi.

Sonraki adımda bir öğrenci işlem kutusuna `A + B` yazmayı önerdi. Hoca toplam sonucunun bir yerde saklanması gerektiğini belirterek bunu `C = A + B` yaptı. Çıktı için de doğrudan `C` yazmak yanlış değildir, ama kullanıcıya mesaj vermek için tek tırnak içinde sabit bir ifade eklenir.

Akış şöyledir:

```
START
  |
  v
çıktı: 'İki sayı giriniz'
  |
  v
girdi: A, B
  |
  v
işlem: C = A + B
  |
  v
çıktı: 'Sonuç=', C
  |
  v
STOP
```

Soru: Çıktıdaki virgül ne için, her zaman kullanmak zorunda mıyız? **Cevap:** Virgül, ekrana yazılacak sabit ifadeden sonra değişkenin geldiğini gösterir. Tek tırnaklı ifade ile değişken arasında bir ayrım gerekir. Hocanın örneği "x. öğrencinin aldığı not" cümlesiydi. Burada x bir değişken, "öğrencinin aldığı" sabit bir ifade, not ise yine bir değişkendir. Değişkenlerle mesajlar arasına her seferinde virgül konur.

20. Örnek 2: Dört İşlem Yapan Algoritma

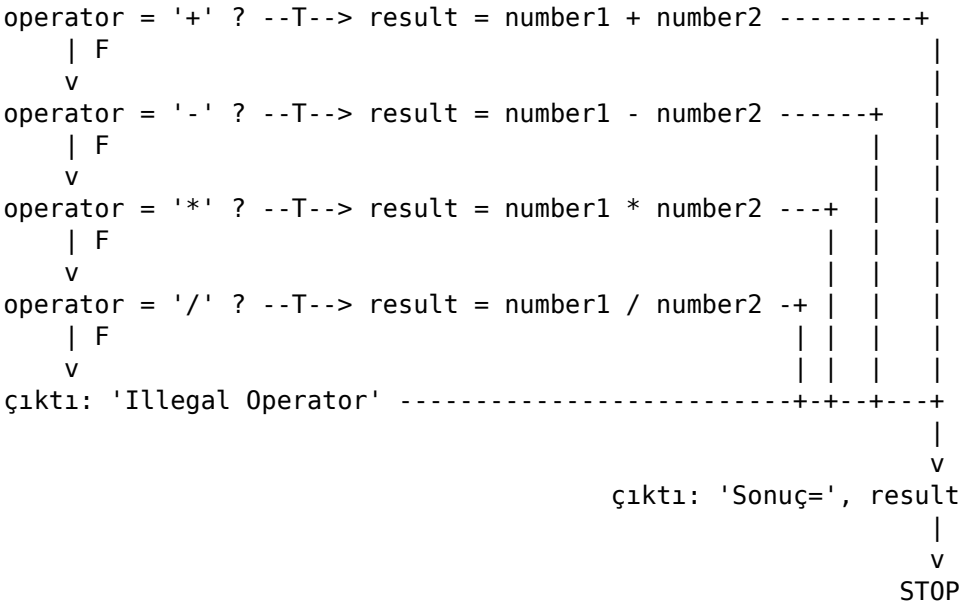
İkinci örnek, dört işlem yapan algoritmanın akış diyagramıdır. Bu örneğin öncekinden farkı, if kutusu gerektirmesidir.

Bir öğrenci işlemleri sayılara atamayı ve bunu çıktı olarak göstermeyi önerdi. Hoca bu yolu seçmedi. Başka bir öğrenci aynı şeyi yeniden sorunca da gerekmediğini söyledi. If içinde işlemin ne olduğu kontrol edilecektir.

Tek if yeterli değildir. Bir kutuda aynı anda dört işlem tipini kontrol etme şansımız yoktur. Her biri için ayrı bir kutu gerekir.

Adımlar şöyledir:

- Ekrana `'İki sayı giriniz'` yazılır.
- `number1` ve `number2` değişkenlerine okuma yapılır.
- Ekrana `'İşlem tipini giriniz'` yazılır.
- `operator` değişkenine okuma yapılır.
- `operator` art arda karşılaştırılır. Karakter sabiti olduğu için `'+'` tek tırnak içinde yazılır.
- Eşleşen koldaki işlem yapılır, hiçbiri eşleşmezse hata mesajı verilir.
- Kollar içeriden dışarıya doğru birleştirilir.
- Ekrana `'Sonuç=', result` yazılır ve STOP.



Son karar kutusunun false kolunda öğrenci önce STOP önerdi, başka bir öğrenci kullanıcıya girilen işlemin tanımlı olmadığının söylenmesini önerdi. Hoca ikincisini kabul etti ve `'Illegal Operator'` mesajını yazdı. Sonra if'leri en içten başlayarak birleştirdi.

> **Ders sonrası not:** Altyazıya göre mesajdan sonraki kol da birleşip sonuç yazdırma adımına gidiyor. Bu durumda `result` hiç değer almadan yazdırılmış olur. Büyük olasılıkla bu kolun doğrudan STOP'a gitmesi daha doğrudur. Bölme kolunda `number2` için sıfır denetimi de derste konuşulmadı.

20.1 Soru-Cevap

Soru: `result` kullanmak yerine sonucu doğrudan çıktı olarak yazsak, örneğin `number1 + number2` diye, olur mu? **Cevap:** Olabilir, yanlış değildir. Ama öğrenme sürecinde sonuçları bir değişkende saklayıp kullanıcıya o değişken üzerinden aktarmak doğru olacaktır.

Soru: `result` değişkeninin varlığını daha önce bir yerde tanımlamamız gerekmez miydi? **Cevap:** Akış diyagramlarında hiçbir değişkeni tanımlamak gerekmez. Değişken yazıldığı yerden itibaren kullanılabilir. Birçok programlama dilinde ise değişkenlerin tanımlanması gerekecektir.

Soru: Aşağı okları hangi durumlarda kullanmamız gerekiyor? **Cevap:** Genelde bu şekilde yazılması gerekir, ama bu bir zorunluluk değildir. Hoca hızlı yazarken bazılarını koymadığını söyledi. Başlangıçta alışkanlık olması için kullanılmasını istiyor. Akışın yönü öğrenildikçe sorun kalmayacaktır.

21. Sonraki Derse Bırakılan Problem

Hoca son olarak bir problem verdi: verilen üç basamaklı bir sayının "kübik sayı" olup olmadığını bulan algoritmanın akış diyagramını çizin. Çözüm bir sonraki derse bırakıldı.

Sınıfa kübik sayının ne olduğu soruldu. Bir öğrenci bir tam sayının küpü dedi, başka bir öğrenci iki kare toplamı olabileceğini söyledi. Hocanın kabul ettiği cevap şuydu: ABC üç basamaklı bir sayı olsun. Basamaklarının küplerinin toplamı kendisine eşitse, yani

$$A^3 + B^3 + C^3 = ABC$$

ise bu sayıya kübik sayı denir. Burada ABC bir çarpım değil, basamakları A, B ve C olan tek bir sayıdır. Hoca böyle bir sayının hemen akla gelmeyeceğini, zaten bu tür işler için bilgisayar kullanmanın akıllıca olduğunu söyledi.

Problemin şartları şunlardır:

- Sayı mutlaka üç basamaklıdır ve kullanıcı bunu bilir. Başka basamak sayıları da incelenebilir, ama şimdilik aranan bu değildir.
- Sayı rakam rakam değil, tek bir sayı olarak verilir. Hocanın örnekleri 125, 722 ve 893 idi.
- Program, sayının bu kuralı sağlayıp sağlamadığını kullanıcıya bir mesajla bildirir.
- Kullanılacak işlemler çarpma, bölme, toplama, çıkarma, `mod`, tam sayı bölmesi ve karşılaştırma işlemleridir. Başka bir şeye ihtiyaç yoktur.

  Ders sonrası not: Hocanın verdiği üç sayı girdinin biçimini gösterir, kuralı sağlamazlar: 125 için $1 + 8 + 125 = 134$, 722 için $343 + 8 + 8 = 359$, 893 için $512 + 729 + 27 = 1268$ (Python ile denetlendi). Bu sayılarda program "sağlamıyor" mesajını vermelidir.

Bir öğrenci bu tür sayılara Armstrong sayısı dendiğini, kübik sayının ise bir sayının küpü olan sayılar için kullanıldığını söyledi. Başka bir öğrenci de 3'ün küpü olan 27'nin kübik sayı olduğunu bildiğini ekledi. Hoca bunu kontrol edeceğini, hata varsa bir sonraki derste düzelteceğini söyledi ve katkı için teşekkür etti. Önemli olan, basamaklarının küpleri toplamı kendisine eşit olan sayıları bulmaktır.

> **Ders sonrası not:** Hoca bu sayılara "kübik sayı" dedi. Öğrencilerin dediği gibi, doğru adlandırma büyük olasılıkla Armstrong sayısıdır (narsisistik sayı da denir). Kübik sayı ya da küp sayı ise büyük olasılıkla bir tam sayının küpü olan sayıdır.

Soru: Tam sayı bölmesi tam olarak neydi, noktadan sonraki kısmı atıyor muydu, yuvarlama var mı? **Cevap:** Evet, noktadan sonraki kısım tamamen atılır, yuvarlama yoktur.

Soru: `div` işlemini arasına doğrudan "div" diye mi yazacağız? Örnek verilebilir mi? **Cevap:** `A div B` diye yazılır. Örneğin `C = 5 div 2` yazılırsa C'nin değeri 2 olur.

Soru: Aynı işlem kutusunda birden fazla işlem yapılabilir mi? **Cevap:** Evet, istediğiniz kadar işlem yazabilirsiniz. Ama genelde 4-5'i geçmemesi istenir.

Soru: Son soruda basamak sayısı belli değil mi, herhangi bir basamaklı sayı olabilir mi? **Cevap:** Hayır. Problem üç basamaklı sayılar için çözülecek.

22. Ders Sonunda Gelen Soru: Sıkıştırma ve Şifreleme

Soru: Sıkıştırma ile şifreleme birebir aynı şey değil, değil mi? **Cevap:** Hayır, ikisi farklıdır. Birinde veri karşı taraf anlamasın diye tamamen karıştırılır. Diğerinde veri daha az bitle ifade edilmeye çalışılır.

Öğrenci, bunu hash algoritmalarıyla karıştırdığını, geri dönüşü olan ve olmayan yöntemlerin bulunduğunu söyledi. Hoca bunu onayladı.

Ek bilgi (derste anlatılmadı): Sıkıştırma ve şifreleme kod çözmeyle geri alınabilirken, hash fonksiyonları büyük olasılıkla tek yönlüdür. Yani hash değerinden veri geri elde edilmek istenmez. Bu nedenle hash büyük olasılıkla ne sıkıştırma ne de şifreleme yöntemi sayılır, daha çok verinin parmak izini üretmek için kullanılır.

Dersin Toparlaması

Dersin ilk bölümünde veri sıkıştırma tamamlandı. Sıkıştırma, kodlama ve kod çözmeden oluşan bir işlemdir. Kod çözme sonunda aynı veri elde ediliyorsa kayıpsız, edilemiyorsa kayıplıdır. RLE tekrarlı veride kazanç sağlar, tekrarsız veride veriyi büyütür. Frekansa bağlı kodlama sık sembollere kısa kodlar verir, göreli kodlama videoda yalnız kareler arası farkı taşır, sözlük kodlaması ise kelimeleri küçük bloklardan oluşturur. Yöntem seçimi verinin örüntüsüne göre yapılır. İletişim hatalarında ise bitlerin değişmesi ele alındı. Eşlik biti tek sayıda bit değişimini tespit eder, çift sayıdaki değişimi kaçırır. Kodlar arası Hamming uzaklığını en az 3 yapmak ise fazladan bit pahasına tek bitlik hatayı en yakın kodu bularak düzeltmeyi sağlar.

Son bölümde algoritma kavramına girildi. Algoritma; sıralı, belirsizlik içermeyen, uygulanabilir ve sonlanan adımlar kümesidir. Bir problemin birden fazla algoritması olabilir ve ilk hedef doğru çalışan bir çözümdür. Derste akış diyagramı sembolleri, işlemler, atama ile karşılaştırma ayrımı ve iki örnek görüldü. Bundan sonra önce akış diyagramı, ardından C kullanılacaktır. Bir sonraki derste, üç basamaklı bir sayının basamaklarının küpleri toplamının kendisine eşit olup olmadığını bulan algoritma çözülecektir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.